Technical Implementation Report: Conformal-Aware Selective Abstention of Zero-Shot Vision-Language Models under Semantic and Visual Distribution Shift

Software Architecture and Implementation Documentation

October 14, 2026

Abstract

This report documents the software architecture, mathematical formulations, data pipelines, caching mechanisms, and verification procedures implemented in the project *Conformal-Aware Selective Abstention of Zero-Shot Vision-Language Models under Semantic and Visual Distribution Shift*. The codebase is supplied as a 59-cell Google Colab Jupyter notebook (`conformal_aware_selective_abstention_clip_colab_persistent_drive.ipynb`) and a synchronized Python script (`conformal_aware_selective_abstention_clip_colab_persistent_drive.py`). The implementation evaluates post-hoc calibration, split-conformal prediction set construction, selective abstention, and out-of-distribution (OOD) detection for a frozen, zero-shot OpenAI CLIP `ViT-B/32` classifier operating over the 10-class CIFAR-10 vocabulary. All calibration parameters and selective operating thresholds are estimated strictly on seeded 5,000-sample subsets of the CIFAR-10 training split, while evaluation spans 30 held-out experimental conditions across clean CIFAR-10, prompt-shifted CIFAR-10, six CIFAR-10-C corruption families at three severity levels, CIFAR-100 semantic OOD, and SVHN far OOD. This report explains how each software component is constructed and how data and control flow through the 26-stage pipeline.

Contents

1 Introduction and Scope **2**

2 Project Structure and Software Dependencies **3**

2.1 Repository File Inventory 3

2.2 External Software and Dataset Dependencies 4

3 Overall Implementation Workflow **4**

4 Input Data and Preparation **6**

4.1 Dataset Specifications and Storage Layout 6

4.2 Verification, Slicing, and Preprocessing 7

5 Main Components **7**

5.1 Utility, Split, and Prompt Components (Sections 2–7) 7

5.2 Inference, Calibration, and Metric Components (Sections 9–16) 8

6 Model and Algorithm Implementation **9**
6.1 Zero-Shot CLIP Similarity and Base Probabilities 9
6.2 Base Confidence and Uncertainty Scores (`MSP`, `entropy`, `margin`) 9
6.3 Post-Hoc Temperature Scaling (`temperature_MSP`) 9
6.4 Multi-Prompt Agreement and Instability (`prompt_agreement`) 10
6.5 Split-Conformal Prediction Sets (`conformal_set_size`) 10

7 Calibration and Inference Processing Procedure **11**
7.1 Two-Stage Caching Architecture 11
7.2 Per-Seed Calibration and Evaluation Workflow 11

8 Configuration and Parameter Handling **13**

9 Evaluation and Postprocessing Implementation **13**
9.1 Fixed-Coverage and Risk-Constrained Selective Prediction 13
9.2 Risk-Coverage Curves, AURC, and EAURC 14
9.3 OOD Discrimination and Calibration Metrics 14
9.4 Bootstrap Confidence Intervals and Failure-Case Extraction 14

10 Output Generation and File Handling **15**

11 Error Handling, Reproducibility, and Execution Considerations **15**
11.1 Validation and Exception Handling 15
11.2 Reproducibility Controls 15

12 Implementation Limitations **17**

13 Summary of the Implementation **17**

A Function and Class Interface Reference **18**

B Notebook Cell Inventory and Execution Map **20**

1 Introduction and Scope

Zero-shot vision-language models such as CLIP classify images by comparing L2-normalized visual and textual embeddings in a shared representation space without task-specific fine-tuning. When deployed under natural prompt variations, visual corruptions, or semantic category shifts, a frozen zero-shot classifier requires post-hoc mechanisms to quantify predictive uncertainty, calibrate confidence estimates, construct finite-sample prediction sets, and abstain on unreliable or out-of-vocabulary inputs.

The supplied codebase implements a self-contained, restart-capable experimental pipeline designed to evaluate six post-hoc confidence and uncertainty scoring rules on top of a frozen OpenAI CLIP `ViT-B/32` model (`openai-clip/ViT-B/32`):

1. **Maximum Softmax Probability (`MSP`)**: Uses the top-1 base softmax probability as confidence and $1 - \text{MSP}$ as uncertainty.
2. **Normalized Shannon Entropy (`entropy`)**: Uses $1 - H_{\text{norm}}(p)$ as confidence and normalized entropy $H_{\text{norm}}(p) \in [0, 1]$ as uncertainty.
3. **Probability Margin (`margin`)**: Uses the difference between the largest and second-largest base softmax probabilities $(p_{(1)} - p_{(2)})$ as confidence and $1 - (p_{(1)} - p_{(2)})$ as uncertainty.
4. **Temperature-Scaled MSP (`temperature_MSP`)**: Scales logits by a scalar temperature $T > 0$ fitted via bounded negative log-likelihood (NLL) minimization on the calibration split before computing top-1 softmax probability.
5. **Multi-Prompt Agreement (`prompt_agreement`)**: Evaluates predictions across four semantically equivalent prompt templates (`P0`–`P3`), using the majority-vote class as the prediction, the fraction of agreeing prompts as confidence, and prompt instability ($1 -$ agreement) as uncertainty.
6. **Split-Conformal Prediction Set Size (`conformal_set_size`)**: Constructs marginal split-conformal prediction sets $C(x) \subseteq \{0, \ldots, 9\}$ at miscoverage level $\alpha = 0.10$ using calibration nonconformity scores $s_i = 1 - p_{\text{base}}(y_i \mid x_i)$, defining confidence as $1/|C(x)|$ and uncertainty as the cardinal set size $|C(x)|$.

The implementation enforces four core design invariants throughout all 26 stages:

- **Frozen zero-shot backbone**: All parameters of CLIP `ViT-B/32` have `requires_grad` set to `False`; no gradient updates or fine-tuning are performed.
- **Fixed 10-class prediction vocabulary**: Every experimental condition—including CIFAR-100 and SVHN OOD inputs—is scored against the 10 canonical CIFAR-10 class names.
- **Strict train-only calibration**: Temperature scaling (T), split-conformal quantiles ($\hat{q}_{1-\alpha}$), fixed-coverage thresholds, and risk-constrained thresholds are fitted exclusively on 5,000-sample subsets drawn from the 50,000-sample CIFAR-10 training split. Held-out test and OOD splits are never accessed during calibration or threshold selection.
- **Fingerprinted persistent caching**: Expensive image and text encoding outputs are cached to disk as compressed `.npz` archives paired with canonical JSON metadata files keyed by 16-character SHA-256 configuration fingerprints.

This report focuses strictly on the software design, mathematical definitions, algorithms, data structures, and verification logic present in the codebase. It does not present empirical findings or numerical performance comparisons.

2 Project Structure and Software Dependencies

2.1 Repository File Inventory

The project is organized around two synchronized source representations of the pipeline and three supporting documentation and audit files:

- `conformal_aware_selective_abstention_clip_colab_persistent_drive.ipynb`: A 59-cell Jupyter notebook (27 Markdown section headers and 32 code cells) structured for execution on Google Colab with Google Drive persistence.

- `conformal_aware_selective_abstention_clip_colab_persistent_drive.py`: A standalone Python script containing the exact code and section structure of the notebook, with IPython `%pip` magic lines commented out for standard Python compatibility.
- `code_changes.py`: A standalone Python module recording all substantive cleanups, consolidations of duplicate function definitions, warning fixes, and retained implementation behaviors.
- `README.md`: User-facing documentation covering setup, execution, and file conventions.
- `report.tex` and `report.pdf`: This technical implementation report.

2.2 External Software and Dataset Dependencies

Table 1 summarizes the Python libraries imported by the implementation and their specific roles in the pipeline.

3 Overall Implementation Workflow

The program is organized as a linear 26-stage pipeline in which each stage consumes validated in-memory objects or persistent disk caches produced by earlier stages:

1. **Stage 1 – Environment and Configuration (Sections 1–2)**: Initializes library imports, mounts Google Drive (when in Colab), defines the global `CONFIG` dictionary, prompt templates `PROMPTS`, corruption list `CORRUPTIONS`, severity list `SEVERITIES`, and the 10-row `EXPERIMENTS` DataFrame, creates output/cache subdirectories under `PERSISTENT_PROJECT_ROOT`, and seeds all RNGs with the first configured seed (`13`).
2. **Stage 2 – Utility Registration (Section 3)**: Registers JSON canonicalization, SHA-256 fingerprinting, array finiteness and probability simplex validators, and atomic `.npz`/`.json` cache read/write helpers.
3. **Stage 3 – Dataset Verification and Split Generation (Sections 4–5)**: Verifies or downloads CIFAR-10, CIFAR-100, SVHN, and CIFAR-10-C; checks array dimensions and label alignment; generates five deterministic 5,000-sample calibration index arrays (`CAL_SPLITS`) from CIFAR-10 train; verifies disjointness against offset test indices (`TEST_INDICES`); and writes `split_metadata.json`.
4. **Stage 4 – Frozen Model and Text/Image Feature Caching (Sections 6–10)**: Loads frozen CLIP `ViT-B/32`, precomputes L2-normalized text embeddings (`TEXT_EMBEDDINGS`) for `P0`–`P3`, wraps datasets in `TorchvisionImageDataset` and `ArrayImageDataset`, and builds `CALIBRATION_BANK` (50,000 CIFAR-10 train samples) and `DATA_BANK` (21 held-out test/OOD datasets) containing validated per-prompt logits, labels, and image embeddings.
5. **Stage 5 – Calibration Fitting and Calibration Feature Caching (Sections 11–17)**: Defines probability, entropy, margin, prompt-agreement, conformal-set, selective-prediction, OOD, calibration, and bootstrap functions. Fits scalar temperature T and conformal quantile $\hat{q}_{1-\alpha}$ per seed on `CALIBRATION_BANK` (`SEED_CALIBRATIONS`), saves `seed_calibration_parameters.json`, and caches per-seed calibration confidence and correctness arrays (`CAL_FEATURES`) for all six methods.

Table 1: Software libraries imported by the implementation and their functional roles.

Package	**Import Symbol(s)**	**Implementation Role**
Python stdlib	`os`, `sys`, `shutil`, `pathlib.Path`, `platform`, `datetime`, `warnings`	Filesystem management, atomic file replacement (`os.replace`), runtime environment inspection, UTC timestamps, and warning filters.
Python stdlib	`json`, `hashlib`, `random`	Canonical JSON serialization, SHA-256 cache fingerprinting, and Python RNG seeding.
Python stdlib	`urllib.request`, `tarfile`, `zipfile`, `math`	Downloading and validating `CIFAR-10-C.tar`, extracting `.npy` members, finite-sample conformal rank calculation (`math.ceil`), and packaging final ZIP archives.
NumPy	`numpy as np`	Array storage, memory-mapped `.npy` reading, vectorized probability/uncertainty math, quantile estimation, trapezoidal integration (`np.trapezoid`), and multinomial/index bootstrap resampling.
pandas	`pandas as pd`	Experiment matrix representation, summary and per-sample DataFrame assembly, grouped aggregations, CSV export, and Markdown table export (`to_markdown`).
SciPy	`scipy.optimize.minimize_scalar`, `scipy.special.logsumexp`	Bounded scalar optimization for temperature scaling and numerically stable log-sum-exp normalization.
scikit-learn	`roc_auc_score`, `average_precision_score`	Binary OOD detection evaluation (AUROC and AUPR-Out).
PyTorch	`torch`, `Dataset`, `DataLoader`	Device management, RNG seeding, inference-mode execution, FP16 automatic mixed precision (`torch.autocast`), and batched image loading.
torchvision	`torchvision`, `datasets`	Downloading and loading canonical CIFAR-10, CIFAR-100, and SVHN splits.
OpenAI CLIP	`clip`	Loading frozen `ViT-B/32` model weights, image preprocessing transforms, BPE text tokenization (`clip.tokenize`), and image/text encoding.
Pillow	`PIL.Image`	Converting raw `uint8` NumPy arrays into RGB `PIL.Image` instances for CLIP preprocessing.
Matplotlib	`matplotlib.pyplot as plt`	Rendering risk-coverage curves, OOD comparisons, reliability diagrams, and heatmaps to PNG and PDF.
tqdm	`tqdm.auto.tqdm`	Progress reporting for downloads, tar extraction, batch encoding, experiment loops, and bootstrap resampling.
Colab API	`google.colab.drive`, `google.colab.files`	Conditional Google Drive mounting and browser download triggers when `IN_COLAB` is `True`.

6. **Stage 6 – Full Experiment Matrix Execution and Statistical Summaries (Sections 18–23)**: Expands `EXPERIMENTS` into the 30-row `FULL_MATRIX`, evaluates all 30 conditions across 5 seeds and 6 methods to produce `summary_df` (`all_results.csv`) and `sample_df` (`per_sample_records.csv`), aggregates fixed-coverage, risk-constrained, calibration, OOD, and curve summary tables, runs 2,000-resample bootstrap confidence intervals (`bootstrap_95ci.csv`), generates 13 figure stems (PNG and PDF), and extracts top-25 diagnostic failure cases per category (`failure_cases.csv`).

7. **Stage 7 – Provenance, Audit, and Packaging (Sections 24–26)**: Writes `experiment_metadata.json`, executes the 34-check `final_validation()` audit (`final_validation_checklist.csv`), copies all deliverables into `final_output/`, writes `completion_summary.json`, and builds downloadable ZIP archives.

4 Input Data and Preparation

4.1 Dataset Specifications and Storage Layout

Table 2 details the datasets used by the pipeline, their split sizes, image dimensions, label conventions, and roles.

Table 2: Datasets used in the implementation, their shapes, and their experimental roles.

Dataset	Split	Samples	Raw Format / Shape	Pipeline Role
CIFAR-10	Train	$50,000$	Python batch files; $(50000, 32, 32, 3)$ `uint8`, labels in $\{0, \ldots, 9\}$	Source pool for the five seeded 5,000-sample calibration subsets (`CAL_SPLITS`).
CIFAR-10	Test	$10,000$	Python batch file; $(10000, 32, 32, 3)$ `uint8`, labels in $\{0, \ldots, 9\}$	Clean ID evaluation (`E1`), prompt-shift ID evaluation (`E2`), and ID reference for OOD detection (`E7`–`E10`).
CIFAR-10-C	Test	$10,000$ per slice	Memory-mapped `<corruption>.npy` of shape $(50000, 32, 32, 3)$ `uint8` and `labels.npy` of shape $(50000,)$	Visual shift evaluation at severities 1 (`E3`), 3 (`E4`, `E6`), and 5 (`E5`) across 6 corruptions (18 slices total).
CIFAR-100	Test	$10,000$	Python batch file; $(10000, 32, 32, 3)$ `uint8`, 100 fine classes	Semantic OOD evaluation under prompt `P0` (`E7`) and prompt `P1` (`E9`).
SVHN	Test	$26,032$	MATLAB `svhn_test.mat`; 32×32 RGB digit crops, labels in $\{1, \ldots, 10\}$	Far-OOD evaluation under prompt `P0` (`E8`) and prompt `P1` (`E10`).

4.2 Verification, Slicing, and Preprocessing

Before loading datasets, `_dataset_present(root, required_paths)` checks whether the underlying extracted files (`data_batch_1`–`5` and `test_batch` for CIFAR-10, `test` for CIFAR-100, and `svhn_test.mat` for SVHN) already exist on persistent storage, setting `download=False` when present and `download=True` otherwise. After loading CIFAR-10, Cell 10 asserts that `cifar10_train_raw.classes` and `cifar10_test_raw.classes` match the canonical 10-element list:

```
CIFAR10_CLASSES = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]
```

For CIFAR-10-C, `extract_cifar10c_tar()` checks for the six required `<corruption>.npy` files (`gaussian_noise`, `motion_blur`, `fog`, `brightness`, `contrast`, `jpeg_compression`) and `labels.npy`. If any file is missing, it validates or downloads `CIFAR-10-C.tar` via `download_with_progress` (writing to a temporary `.part` file and atomically renaming upon completion) and extracts only the seven required files. Cell 12 then opens each `.npy` file in read-only memory-mapped mode (`mmap_mode="r"`), asserts shape $(50000, 32, 32, 3)$, and verifies that for every severity $s \in \{1, 3, 5\}$, the 10,000-element label slice `labels_c[(s - 1) * 10000 : s * 10000]` is elementwise identical to `cifar10_test_raw.targets`.

Image preprocessing is handled by two `torch.utils.data.Dataset` subclasses in Section 8:

- `ArrayImageDataset(images, labels, indices=None)`: Indexes into a memory-mapped CIFAR-10-C slice, casts the $(32, 32, 3)$ array to `uint8`, constructs a `PIL.Image.fromarray(..., mode="RGB")`, applies the OpenAI CLIP `preprocess` pipeline (bicubic resize/center-crop to 224×224, RGB tensor conversion, and channel-wise CLIP mean/std normalization), and returns the 3-tuple `(tensor, int(label), int(idx))`.
- `TorchvisionImageDataset(base_dataset, label_transform=None)`: Wraps a `torchvision` dataset, converts non-PIL images to `uint8` RGB `PIL.Image` objects if needed, applies CLIP `preprocess`, and applies `label_transform(y)`. For SVHN, `svhn_dataset()` supplies `label_transform=lambda x: 0 if int(x) == 10 else int(x)` to map SVHN's label `10` (digit zero) to `0` for metadata consistency (note that OOD evaluation ignores ground-truth class labels and sets `correctness = np.nan`).

5 Main Components

Table **??** provides a structured inventory of the classes and functions implemented across the notebook and script, detailing their inputs, return values, and responsibilities.

5.1 Utility, Split, and Prompt Components (Sections 2–7)

- `set_global_seed(seed: int) -> None` and `seed_numpy(seed: int) -> np.random.Generator`: `set_global_seed` seeds `random`, `np.random`, and `torch` (CPU and all CUDA devices), sets `cudnn.deterministic = True` and `cudnn.benchmark = False`, and calls `torch.use_deterministic_algorithms(True, warn_only=True)`. `seed_numpy` returns an isolated `np.random.default_rng(seed)` instance so calibration split generation does not depend on global RNG state.
- `canonical_json(obj) -> str`, `sha256_text(text: str) -> str`, and `config_fingerprint(extra: dict) -> str`: Serialize nested dictionaries with sorted keys and

compact separators (`(",", ":")`), hash the resulting UTF-8 string with SHA-256, and slice the first 16 hex characters to version cache files.

- `save_array_cache(path: Path, arrays: dict, metadata: dict) -> None` and `load_array_cache(path: Path, expected_fingerprint: str)`: `save_array_cache` writes `arrays` to a temporary `.tmp.npz` file, atomically moves it to `path` via `os.replace`, and writes `metadata` to `<path>.json`. `load_array_cache` reads `<path>.json`, verifies `metadata.get("fingerprint") == expected_fingerprint`, and loads the `.npz` archive with `allow_pickle=False`. If the file is missing, corrupt, or has a mismatched fingerprint, it returns `None`.

- `make_calibration_split(seed: int, n_total: int = 50000, n_cal: int = 5000) -> np.ndarray`: Validates $n_{\text{total}} > 1$ and $0 < n_{\text{cal}} < n_{\text{total}}$, samples n_{cal} unique indices from $[0, n_{\text{total}})$ using `seed_numpy(seed).choice(..., replace=False)`, sorts them in ascending order as `int64`, and asserts 1-D shape, exact length, uniqueness, and bounds.

- `build_prompts(class_names, prompt_key)` and `get_text_embeddings(prompt_key)`: Format each class name into the template `PROMPTS[prompt_key]` (e.g., `"a photo of a airplane."`), tokenize the 10 strings via `clip.tokenize`, encode them with `model.encode_text(tokens)`, normalize each row by its L2 norm clamped at 10^{-12}, cache the $(10, 512)$ `float32` array, and return it.

5.2 Inference, Calibration, and Metric Components (Sections 9–16)

- `encode_image_dataset(dataset, cache_name, expected_extra) -> tuple[np.ndarray, np.ndarray]`: Checks for a valid cached image-embedding archive; on a cache miss, iterates over a non-shuffled `DataLoader` of size `CONFIG["batch_size"]`, runs `model.encode_image(images)` (inside `torch.autocast("cuda", dtype=torch.float16)` when `CONFIG["use_amp"]` is true), normalizes embeddings to unit L2 norm, concatenates batches into $(N, 512)$ `float32` embeddings and $(N,)$ `int64` labels, caches them, and returns `(emb, y)`.

- `compute_prompt_logits(image_embeddings, prompt_embeddings) -> dict[str, np.ndarray]`: Retrieves the scalar logit scale $s = \exp(\texttt{model.logit_scale})$ and computes $Z^{(p)} = s \cdot E_{\text{img}}(E_{\text{txt}}^{(p)})^\top \in \mathbb{R}^{N \times 10}$ for each prompt $p \in \{\texttt{P0}, \texttt{P1}, \texttt{P2}, \texttt{P3}\}$.

- `_validate_prompt_logits_bank` and `get_condition_logits`: Verify that all four prompts exist in the logits dictionary, shapes equal $(N, 10)$, label length equals embedding length N, and all arrays are finite. Cache logits under keys `logits_P0`–`logits_P3`.

- `evaluate_seed_condition(seed, condition_id, logits_by_prompt, labels, primary_prompt, is_ood)`: Computes all six methods' predictions, confidences, uncertainties, and conformal set sizes for one condition and seed. For in-distribution conditions (`not is_ood`), evaluates fixed-coverage operating points ($\{0.50, 0.70, 0.80, 0.90\}$), risk-constrained operating points ($\{0.05, 0.10\}$), curve summaries (full-coverage risk, AURC, EAURC), and calibration metrics (ECE, NLL, Brier). Returns `(summary_rows_df, per_method_dict, extras_dict)`.

6 Model and Algorithm Implementation

6.1 Zero-Shot CLIP Similarity and Base Probabilities

For an input image x_i, let $z_{\text{img}}(x_i) \in \mathbb{R}^D$ ($D = 512$ for `ViT-B/32`) denote the raw visual feature vector produced by `model.encode_image`, and for class $c \in \{0, \ldots, K-1\}$ ($K = 10$) under prompt template $p \in \{\texttt{P0}, \texttt{P1}, \texttt{P2}, \texttt{P3}\}$, let $z_{\text{txt}}^{(p)}(c) \in \mathbb{R}^D$ denote the raw text feature vector produced by `model.encode_text`. Both vectors are L2-normalized with a lower norm clamp of $\varepsilon = 10^{-12}$:

$$\hat{u}_i = \frac{z_{\text{img}}(x_i)}{\max(\|z_{\text{img}}(x_i)\|_2,\, 10^{-12})}, \qquad \hat{v}_c^{(p)} = \frac{z_{\text{txt}}^{(p)}(c)}{\max\left(\|z_{\text{txt}}^{(p)}(c)\|_2,\, 10^{-12}\right)}. \tag{1}$$

With learned scalar logit scale $s = \exp(\texttt{model.logit_scale})$, the zero-shot logit for sample i and class c under prompt p is:

$$\ell_{i,c}^{(p)} = s\, \langle \hat{u}_i,\, \hat{v}_c^{(p)} \rangle. \tag{2}$$

In `softmax_np(logits)`, logits are cast to `float64`, shifted by logsumexp($\ell_i^{(p)}$) for numerical stability, exponentiated, and normalized to form the probability vector $p_i^{(p)} \in \Delta^{K-1}$:

$$p_{i,c}^{(p)} = \frac{\exp\left(\ell_{i,c}^{(p)} - \log \sum_{k=0}^{K-1} \exp(\ell_{i,k}^{(p)})\right)}{\sum_{j=0}^{K-1} \exp\left(\ell_{i,j}^{(p)} - \log \sum_{k=0}^{K-1} \exp(\ell_{i,k}^{(p)})\right)}. \tag{3}$$

6.2 Base Confidence and Uncertainty Scores (`MSP`, `entropy`, `margin`)

Given a primary prompt $p \in \{\texttt{P0}, \texttt{P1}\}$ for an experimental condition, let $p_i = p_i^{(p)}$ denote the base probability vector and let $p_{i,(1)} \geq p_{i,(2)} \geq \cdots \geq p_{i,(K)}$ denote its sorted components in descending order (`top12_stats`). The predicted class is $\hat{y}_i = \arg\max_c p_{i,c}$. The three base scoring methods are defined as follows:

- **MSP**:

$$\text{conf}_{\text{MSP}}(x_i) = p_{i,(1)}, \qquad \text{unc}_{\text{MSP}}(x_i) = 1 - p_{i,(1)}. \tag{4}$$

- **Normalized Entropy** (with probability clipping $\tilde{p}_{i,c} = \min(\max(p_{i,c}, 10^{-12}), 1.0)$ in `entropy_np`):

$$\begin{aligned} H_{\text{norm}}(p_i) &= -\frac{1}{\ln K} \sum_{c=0}^{K-1} \tilde{p}_{i,c} \ln \tilde{p}_{i,c}, \\ \text{conf}_{\text{ent}}(x_i) &= 1 - H_{\text{norm}}(p_i), \qquad \text{unc}_{\text{ent}}(x_i) = H_{\text{norm}}(p_i). \end{aligned} \tag{5}$$

- **Margin**:

$$\text{conf}_{\text{margin}}(x_i) = p_{i,(1)} - p_{i,(2)}, \qquad \text{unc}_{\text{margin}}(x_i) = 1 - \left(p_{i,(1)} - p_{i,(2)}\right). \tag{6}$$

6.3 Post-Hoc Temperature Scaling (`temperature_MSP`)

For each seed s and its calibration index set $\mathcal{I}_{\text{cal}}^{(s)} \subset \{0, \ldots, 49999\}$ of size $n = 5000$, `fit_temperature` finds a scalar temperature $T^{(s)} \in [0.05, 10.0]$ minimizing the empirical multiclass negative log-likelihood on the calibration prompt `P0`:

$$T^{(s)} = \arg \min_{T \in [0.05, 10.0]} \frac{1}{n} \sum_{i \in \mathcal{I}_{\text{cal}}^{(s)}} \left(\log \sum_{c=0}^{K-1} \exp\left(\frac{\ell_{i,c}^{(\texttt{P0})}}{T}\right) - \frac{\ell_{i,y_i}^{(\texttt{P0})}}{T} \right), \tag{7}$$

using Brent's bounded scalar minimization (`scipy.optimize.minimize_scalar(..., method="bounded", options={"xatol": 1e-4, "maxiter": 200})`). At evaluation time on primary prompt p, `temperature_scale_logits` computes $\tilde{\ell}_i^{(p)} = \ell_i^{(p)}/T^{(s)}$, and `temperature_MSP` uses the resulting top-1 softmax probability $\tilde{p}_{i,(1)}^{(p)}$ as confidence and $1-\tilde{p}_{i,(1)}^{(p)}$ as uncertainty. Because $T^{(s)} > 0$, temperature scaling preserves the top-1 argmax class prediction.

6.4 Multi-Prompt Agreement and Instability (`prompt_agreement`)

Let $\mathcal{P} = \{\texttt{P0}, \texttt{P1}, \texttt{P2}, \texttt{P3}\}$ denote the four prompt templates ($M = |\mathcal{P}| = 4$). For each prompt $p \in \mathcal{P}$, the predicted class for sample i is $\hat{y}_i^{(p)} = \arg\max_c \ell_{i,c}^{(p)}$. In `prompt_level_outputs`, class vote counts are accumulated across the M prompts:

$$v_{i,c} = \sum_{p \in \mathcal{P}} \mathbb{I}\Big(\hat{y}_i^{(p)} = c\Big), \qquad c \in \{0, \ldots, K-1\}. \tag{8}$$

The majority-vote prediction, prompt agreement confidence, and prompt instability uncertainty are defined as:

$$\begin{aligned} \hat{y}_i^{\text{maj}} &= \arg\max_{c \in \{0,\ldots,K-1\}} v_{i,c}, \\ \text{conf}_{\text{PA}}(x_i) &= \frac{v_{i,\hat{y}_i^{\text{maj}}}}{M} \in \{0.25, 0.50, 0.75, 1.00\}, \qquad \text{unc}_{\text{PA}}(x_i) = 1 - \text{conf}_{\text{PA}}(x_i), \end{aligned} \tag{9}$$

where ties in $\arg\max_c v_{i,c}$ are broken by the lowest class index c (`np.argmax` convention).

6.5 Split-Conformal Prediction Sets (`conformal_set_size`)

For each calibration sample $i \in \mathcal{I}_{\text{cal}}^{(s)}$ with true label y_i and unscaled `P0` softmax probability vector $p_i^{(\texttt{P0})}$, `fit_seed_calibration` defines the nonconformity score:

$$s_i = 1 - p_{i,y_i}^{(\texttt{P0})} \in [0, 1]. \tag{10}$$

Given target miscoverage $\alpha = 0.10$ and calibration size $n = 5000$, the finite-sample corrected order-statistic rank and quantile level are:

$$r = \lceil (n+1)(1-\alpha) \rceil = \lceil 5001 \times 0.90 \rceil = 4501, \qquad q_{\text{level}} = \min\left(1.0,\, \frac{r}{n}\right) = \frac{4501}{5000} = 0.9002, \tag{11}$$

and $\hat{q}_{1-\alpha}^{(s)}$ is computed as `np.quantile(conformal_nonconf, quantile_level, method="higher")`, which selects the r-th smallest nonconformity score $s_{(r)}$.

At evaluation time on primary prompt p with unscaled probability vector $p_i^{(p)}$, `conformal_sets` constructs the prediction set:

$$C(x_i) = \left\{ c \in \{0, \ldots, K-1\} : p_{i,c}^{(p)} \geq (1 - \hat{q}_{1-\alpha}^{(s)}) - 10^{-12} \right\}. \tag{12}$$

If $C(x_i)$ is empty for any sample, the top-1 class $\arg\max_c p_{i,c}^{(p)}$ is inserted so that $|C(x_i)| \geq 1$ holds everywhere. The `conformal_set_size` method uses the base argmax prediction $\hat{y}_i = \arg\max_c p_{i,c}^{(p)}$ as its point prediction and defines:

$$\text{conf}_{\text{conf}}(x_i) = \frac{1}{\max(|C(x_i)|, 1)}, \qquad \text{unc}_{\text{conf}}(x_i) = |C(x_i)| \in \{1, \ldots, 10\}. \tag{13}$$

7 Calibration and Inference Processing Procedure

7.1 Two-Stage Caching Architecture

Because encoding images through CLIP `ViT-B/32` across 22 dataset splits (50,000 train images + 10,000 CIFAR-10 test + 10,000 CIFAR-100 test + 26,032 SVHN test + $18 \times 10,000$ CIFAR-10-C images = $276,032$ total images) is the primary computational bottleneck, the pipeline decouples feature extraction into three cached stages:

1. **Text Embedding Cache (`cache/text_embeddings/`)**: Encodes the 10 class prompts for each template `P0`–`P3` once and saves $(10,512)$ normalized embeddings.
2. **Image Embedding Cache (`cache/image_embeddings/`)**: Encodes each dataset split once (independent of prompt templates) and saves $(N,512)$ normalized image embeddings and $(N,)$ labels.
3. **Logit Bank Cache (`cache/logits/`)**: Multiplies cached image embeddings by all four cached text embedding matrices, scales by exp(`model.logit_scale`), validates shapes and finiteness via `_validate_prompt_logits_bank`, and stores `logits_P0`–`logits_P3`.

7.2 Per-Seed Calibration and Evaluation Workflow

Once `CALIBRATION_BANK` and `DATA_BANK` are populated in Section 10:

1. Section 12 iterates over `seed` in `[13, 42, 123, 456, 765]`, slices `CALIBRATION_BANK` using `CAL_SPLITS[seed]`, fits $T^{(s)}$ and $\hat{q}_{1-\alpha}^{(s)}$, and stores the resulting dictionary in `SEED_CALIBRATIONS[seed]`.
2. Section 17 iterates over `SEED_CALIBRATIONS`, evaluates all six uncertainty methods on the 5,000 calibration samples for each seed, and stores the calibration confidence vectors (`CAL_FEATURES[seed]["conf"][method]`) and boolean correctness vectors (`CAL_FEATURES[seed]["correct"][method]`).
3. Section 18 precomputes clean CIFAR-10 (`E1`) ID uncertainty arrays per seed and prompt in `id_seed_cache`, then iterates over the 30 rows of `FULL_MATRIX` and the 5 seeds. For each `(row, seed)` pair:
   - `evaluate_seed_condition` computes test-set predictions, confidences, and uncertainties for all 6 methods.
   - For ID conditions (`E1`–`E6`), `evaluate_seed_condition` selects thresholds from `CAL_FEATURES[seed]` for the 4 fixed coverage targets and 2 risk targets, evaluates test-set selective metrics, computes AURC/EAURC, and computes ECE/NLL/Brier.
   - For OOD conditions (`E7`–`E10`), the outer loop pairs the OOD uncertainty scores with the corresponding clean CIFAR-10 ID uncertainty scores from `id_seed_cache[seed][prompt_condition]` and calls `ood_metrics(id_u, ood_u)`.
   - Six per-sample DataFrames (one per method, each of length N) are concatenated and appended to `all_sample_records`.

Table 3: Configuration parameters in `CONFIG`, their types, values, and effects.

Key	Type	Configured Value	Effect in Pipeline
`model_name`	`str`	`"ViT-B/32"`	Architecture name passed to `clip.load`.
`model_identifier`	`str`	`"openai-clip/ViT-B/32"`	Identifier included in `config_fingerprint` and metadata.
`seeds`	`list[int]`	`[13, 42, 123, 456, 765]`	Seeds controlling calibration split sampling and bootstrap RNG offsets.
`calibration_size`	`int`	`5000`	Number of CIFAR-10 train samples per seed calibration subset.
`batch_size`	`int`	`128` (CUDA) / `32` (CPU)	Mini-batch size for `DataLoader` during CLIP image encoding.
`num_workers`	`int`	`min(4, cpu_count)` $\rightarrow$ `0`	Set in Cell 05, then overridden to `0` in Cell 25 for Colab stability.
`pin_memory`	`bool`	`torch.cuda.is_available()`	Enables pinned host memory in `DataLoader` when on CUDA.
`use_amp`	`bool`	`torch.cuda.is_available()`	Enables FP16 `torch.autocast` during `model.encode_image` on CUDA.
`temperature_bounds`	`tuple`	`(0.05, 10.0)`	Lower and upper bounds for scalar temperature optimization in `fit_temperature`.
`conformal_alpha`	`float`	`0.10`	Target miscoverage level α for split-conformal calibration ($1 - \alpha = 0.90$).
`coverage_targets`	`list[float]`	`[0.50, 0.70, 0.80, 0.90]`	Nominal coverage levels for fixed-coverage selective prediction.
`risk_targets`	`list[float]`	`[0.05, 0.10]`	Maximum calibration selective risk bounds for risk-constrained coverage.
`bootstrap_resamples`	`int`	`2000`	Number of bootstrap replicates per confidence interval (enforced equal to 2,000).
`bootstrap_seed`	`int`	`20260915`	Base seed offset added to `seed` (plus `0`, `17`, or `101`) in bootstrap functions.
`ece_bins`	`int`	`15`	Number of equal-width bins in $[0, 1]$ for ECE and reliability diagrams.
`cache_enabled`	`bool`	`True`	Controls whether image embeddings and logits are read from and saved to disk.
`cache_version`	`str`	`"v2_persistent_drive"`	Version tag included in `config_fingerprint` to invalidate incompatible

8 Configuration and Parameter Handling

Table 3 documents all keys in the central `CONFIG` dictionary (defined in Cell 05 and updated in Cell 25), their values, and their effects on execution.

Table 4 lists the 10 experiment families (`E1`–`E10`) defined in `EXPERIMENTS` and expanded by `expanded_experiment_matrix()` into the 30 rows of `FULL_MATRIX`.

Table 4: Experiment families `E1`–`E10` and their expansion in `FULL_MATRIX`.

ID	Dataset	Shift Type	Sev.	Prompt	OOD	Conds.	Condition ID Format
E1	CIFAR-10	none	0	P0	False	1	E1
E2	CIFAR-10	prompt	0	P1	False	1	E2
E3	CIFAR-10-C	visual	1	P0	False	6	E3::<corruption>::s1
E4	CIFAR-10-C	visual	3	P0	False	6	E4::<corruption>::s3
E5	CIFAR-10-C	visual	5	P0	False	6	E5::<corruption>::s5
E6	CIFAR-10-C	visual+prompt	3	P1	False	6	E6::<corruption>::s3
E7	CIFAR-100	semantic	0	P0	True	1	E7
E8	SVHN	far_ood	0	P0	True	1	E8
E9	CIFAR-100	semantic+prompt	0	P1	True	1	E9
E10	SVHN	far_ood+prompt	0	P1	True	1	E10

9 Evaluation and Postprocessing Implementation

9.1 Fixed-Coverage and Risk-Constrained Selective Prediction

For any uncertainty method m with calibration confidence vector $c_{\text{cal}} \in \mathbb{R}^{5000}$ and calibration correctness indicators $y_{\text{cal}} \in \{0,1\}^{5000}$:

- **Fixed-coverage threshold (`threshold_for_target_coverage`)**: For target coverage $\tau \in \{0.50, 0.70, 0.80, 0.90\}$, the threshold is the $(1-\tau)$ empirical quantile of c_{cal} using `method="lower"`:
$$\hat{t}(\tau) = \text{Quantile}_{1-\tau}^{\text{lower}}(c_{\text{cal}}). \tag{14}$$
- **Risk-constrained threshold (`max_coverage_under_risk`)**: For target risk $r_{\max} \in \{0.05, 0.10\}$, calibration samples are sorted by descending confidence using a stable mergesort (`np.argsort(-c, kind="mergesort")`). Because selecting all samples with $c_i \geq t$ always includes entire tie blocks of identical confidence values, the function identifies the last index e_b of every distinct tie block b, computes the cumulative error count $E(e_b) = \sum_{j=0}^{e_b}(1 - y_{\text{sorted},j})$ and prefix size $k_b = e_b + 1$, and computes the tie-block empirical risk:
$$\hat{R}_b = \frac{E(e_b)}{k_b}. \tag{15}$$
If at least one block satisfies $\hat{R}_b \leq r_{\max} + 10^{-12}$, the function returns the confidence value at the largest valid block index b^* (maximizing calibration coverage). If no block satisfies the constraint, it returns nextafter$(\max(c_{\text{cal}}), +\infty)$, which selects zero samples.
- **Test-set selective evaluation (`selective_metrics`)**: Given test confidences c_{test}, correctness indicators y_{test}, and calibration threshold $\hat{t}$, the selected mask is $S = \{i : c_{\text{test},i} \geq \hat{t}\}$. Realized coverage is $|S|/N_{\text{test}}$, abstention rate is $1 -$ coverage, and selective risk is $1 - \frac{1}{|S|}\sum_{i \in S} y_{\text{test},i}$ (or `np.nan` if $|S| = 0$).

9.2 Risk-Coverage Curves, AURC, and EAURC

In `risk_coverage_curve(confidence, correct)`, test samples are sorted by descending confidence via `kind="mergesort"`. For each prefix size $k \in \{1, \ldots, N\}$, the prefix coverage and prefix risk are:

$$\text{cov}_k = \frac{k}{N}, \qquad \text{risk}_k = \frac{1}{k} \sum_{j=1}^{k} (1 - y_{\text{sorted},j}) \,, \tag{16}$$

with $(0.0, 0.0)$ prepended at the origin. `aurc` integrates the curve via `np.trapezoid(risk, coverage)`. `oracle_aurc` computes the AURC when confidence is replaced by the binary correctness indicator $y \in \{0.0, 1.0\}$ (ranking all correct predictions ahead of all errors), and `eaurc` returns AURC − Oracle AURC.

9.3 OOD Discrimination and Calibration Metrics

For OOD conditions (`E7`–`E10`), clean CIFAR-10 (`E1`) test samples are labeled 0 (in-distribution) and OOD samples are labeled 1 (out-of-distribution), with higher uncertainty scores indicating higher OOD likelihood (`score_orientation = "higher_is_more_ood"`). `ood_metrics(id_u, ood_u)` computes:

- **AUROC**: `roc_auc_score(y, s)` over the concatenated binary labels y and uncertainty scores s.
- **AUPR-Out**: `average_precision_score(y, s)` with OOD as the positive class.
- **FPR95 (`fpr95`)**: Sets the threshold $\gamma_{0.05}$ to the 5th percentile of OOD uncertainty scores (`np.quantile(ood_u, 0.05, method="linear")`) so that approximately 95% of OOD inputs satisfy ood_u $\geq \gamma_{0.05}$, and returns the fraction of ID inputs with id_u $\geq \gamma_{0.05}$.

For in-distribution calibration evaluation on N samples with probability matrix $P \in [0, 1]^{N \times K}$ and true labels $y \in \{0, \ldots, K-1\}^N$:

- **ECE (`ece_score`)**: Partitions $(0, 1]$ into $B = 15$ equal-width bins $(b_{m-1}, b_m]$ and computes $\sum_{m=1}^{B} \frac{|B_m|}{N} |\bar{c}(B_m) - \bar{a}(B_m)|$.
- **NLL (`nll_score`)**: Computes $-\frac{1}{N} \sum_{i=1}^{N} \ln \max(P_{i,y_i}, 10^{-12})$.
- **Multiclass Brier Score (`brier_score_multiclass`)**: Computes $\frac{1}{N} \sum_{i=1}^{N} \sum_{c=0}^{K-1} (P_{i,c} - \mathbb{I}(y_i = c))^2$.

9.4 Bootstrap Confidence Intervals and Failure-Case Extraction

Section 20 (Cell 46) computes 2,000-resample 95% percentile bootstrap confidence intervals ($[q_{0.025}, q_{0.975}]$) for:

- `risk_at_90` and `aurc` across primary ID conditions `E1`, `E2`, `E4`, `E5`, `E6` (5 conditions × 5 seeds × 6 methods × 2 metrics = 300 CIs). Note that for `E4`, `E5`, and `E6`, `sample_df[sample_df.experiment_id == exp_id]` pools all 6 CIFAR-10-C corruptions ($N = 60{,}000$ samples per seed and method).
- `auroc` across primary OOD conditions `E7` and `E8` (2 conditions × 5 seeds × 6 methods = 60 CIs), independently resampling $n_{\text{id}} = 10{,}000$ ID scores and n_{ood} OOD scores with replacement on each replicate.

Section 22 (Cell 50) defines `extract_failure_cases(sample_df, n_each=25)`, which extracts the top 25 rows per `(experiment_id, method)` group across six diagnostic categories: `high_confidence_wrong` (ID errors with highest confidence), `low_confidence_correct` (ID correct predictions with lowest confidence), `prompt_instability` (highest `prompt_instability`), `large_conformal_set` (largest `conformal_set_size`), `ood_overconfidence` (OOD samples with lowest uncertainty), and `shift_sensitive_behavior` (shifted ID samples in `E2`–`E6` with the largest confidence drop relative to the same `(seed, method, sample_id)` in `E1`), deduplicating the combined table and exporting 6,150 rows to `failure_cases.csv`.

10 Output Generation and File Handling

Table 5 summarizes all files written by the pipeline across `output/`, `tables/`, `figures/`, `metadata/`, and `final_output/`.

11 Error Handling, Reproducibility, and Execution Considerations

11.1 Validation and Exception Handling

The codebase uses fail-fast exception handling for data and numerical invariants and defensive boolean aggregation for the final pipeline audit:

- **Numerical finiteness and simplex validation**: `assert_finite` and `validate_probabilities` raise `ValueError` immediately if any embedding, logit, probability vector, or nonconformity score contains `NaN`/`Inf` or if softmax rows deviate from 1.0 by more than 10^{-5}.
- **Cache corruption recovery**: `load_array_cache` and `get_condition_logits` catch exceptions from unreadable or schema-mismatched cache files, log the reason, and recompute and overwrite the cache atomically via temporary `.tmp.npz` files.
- **34-check integrated audit (`final_validation`)**: Section 25 evaluates 34 boolean checks covering Python version (≥ 3.9), device resolution, zero trainable model parameters, prompt keys, calibration split sizes and train/test disjointness, dataset/experiment/method coverage, result schema, finiteness of embeddings/logits/selective metrics/OOD metrics/calibration metrics/bootstrap CIs/conformal set sizes, row uniqueness in `summary_df` and `sample_df`, and presence of all required tables, figures, and metadata files. Cell 58 blocks final export if any check in `validation_df["passed"]` is `False`.

11.2 Reproducibility Controls

Reproducibility is controlled at four levels:

1. **Global RNG seeding**: `set_global_seed(13)` seeds Python, NumPy, and PyTorch RNGs and enables deterministic CuDNN and PyTorch algorithm flags.
2. **Isolated split generators**: `make_calibration_split(seed)` instantiates an isolated `np.random.default_rng(seed)` for each seed in `[13, 42, 123, 456, 765]`, ensuring that calibration indices depend only on `seed` and not on prior random calls.

Table 5: Generated output files, formats, and directory locations.

Directory	Filename(s)	Description and Schema
output/	all_results.csv	6,360-row condition-level summary table with 25 standardized columns (RESULT_COLUMNS).
output/	per_sample_records.csv	9,961,920-row sample-level table recording predictions, confidences, uncertainties, prompt vote strings, conformal set sizes, and correctness.
tables/	risk_at_90_coverage.csv, risk_at_80_coverage.csv, risk_at_70_coverage.csv, risk_at_50_coverage.csv	Mean, standard deviation, and seed count of selective risk at nominal 90%, 80%, 70%, and 50% coverage.
tables/	risk_constrained_coverage.csv	Mean, standard deviation, and seed count of realized coverage under calibration risk bounds 5% and 10%.
tables/	calibration_metrics.csv, ood_metrics.csv, curve_metrics.csv	Grouped mean and standard deviation across seeds for ECE/NLL/Brier, AUROC/AUPR-Out/FPR95, and AURC/EAURC/full-risk.
tables/	main_method_comparison.csv, robustness_table.csv	Primary comparison tables for 90%-coverage selective behavior and full risk-coverage curve summaries.
tables/	bootstrap_95ci.csv, failure_cases.csv	360-row 95% bootstrap CI table and 6,150-row diagnostic failure-case table (plus 8 companion .md tables).
figures/	*.png, *.pdf	13 figure stems exported in both 200-DPI raster PNG and vector PDF formats (26 files total).
metadata/	split_metadata.json, seed_calibration_parameters.json, experiment_metadata.json, final_validation_checklist.csv	Provenance, calibration parameters, split definitions, and the 34-row validation audit table.
final_output/	Mirror of figures/, tables/, metadata/, CSVs, and completion_summary.json	Consolidated export directory archived into /content/conformal_vlm_research_artifacts.zip.

3. **Deterministic tie-breaking**: All sorting operations in selective prediction and AURC calculation specify `kind="mergesort"` so tied confidence scores preserve sample order deterministically.

4. **Deterministic bootstrap seeds**: Bootstrap functions use isolated `np.random.default_rng` instances seeded with `BOOTSTRAP_SEED + seed` (for risk at 90%), `BOOTSTRAP_SEED + seed + 17` (for AURC), and `BOOTSTRAP_SEED + seed + 101` (for OOD AUROC).

Setting random seeds does not guarantee bit-identical floating-point logits across different GPU

architectures or CUDA/cuDNN versions, particularly when FP16 automatic mixed precision (`use_amp=True`) is active during `model.encode_image`. Once embeddings and logits are cached in `cache/`, however, all downstream calibration, selective prediction, OOD evaluation, and bootstrap computations are deterministic on CPU.

12 Implementation Limitations

The following limitations and design constraints follow directly from inspection of the supplied source code:

1. **Unscaled probabilities in conformal set construction**: In `fit_seed_calibration` (Section 12) and `evaluate_seed_condition` (Section 16), conformal nonconformity scores and prediction sets $C(x)$ are computed from the unscaled base softmax probabilities (`base["probs"]`) rather than temperature-scaled probabilities (`temp["probs"]`).

2. **Coarse discreteness of `prompt_agreement` and `conformal_set_size`**: Because `prompt_agreement` takes at most 4 distinct values ($\{0.25, 0.50, 0.75, 1.00\}$) and `conformal_set_size` confidence $1/|C(x)|$ takes at most 10 distinct values, threshold selection via confidence $\geq \hat{t}$ selects entire tie blocks at once. Consequently, realized coverage can differ noticeably from nominal target coverage τ.

3. **Seed and method pooling in Section 21 visualizations**: In Cell 48, calling `plot_risk_coverage("E1")` and `plot_risk_coverage("E2")` with the default `seed=None` (as well as the `E4` corruption plots) passes all 5 seeds' rows ($50,000$ rows per method) into `risk_coverage_curve`. In the same cell, `prompt_agreement_vs_accuracy` and `conformal_set_size_vs_correctness` group `sample_df[sample_df.experiment_id == "E1"]` without filtering to a single `method` per sample, averaging five primary-prompt correctness indicators with one majority-vote correctness indicator per sample.

4. **Unused helper functions retained for interface compatibility**: `evaluate_condition_experiment` in Cell 39 and the un-instrumented bootstrap helpers (`bootstrap_ci_scalar`, `bootstrap_risk_ci`, `bootstrap_aurc_ci`) in Cell 35 are defined in the codebase, while Cell 42 and Cell 46 invoke `get_bank_key` and the progress-aware bootstrap functions (`bootstrap_risk_ci_progress`, `bootstrap_aurc_ci_progress`, `bootstrap_auroc_ci_progress`), respectively.

5. **High memory footprint of `sample_df`**: Storing $9,961,920$ rows in `sample_df` before writing `per_sample_records.csv` requires several gigabytes of system RAM.

6. **Hardcoded `/content` paths outside Colab**: When `IN_COLAB` is `False`, `MY_DRIVE` defaults to `Path("/content")` and Cell 58 writes ZIP archives to `/content/*.zip`, which requires write permissions on `/content` unless modified.

13 Summary of the Implementation

The project implements a complete, cached evaluation pipeline for zero-shot CLIP `ViT-B/32` uncertainty quantification, selective abstention, and OOD detection across 30 experimental conditions and 5 seeds. By separating image and text embedding extraction from downstream statistical evaluation through fingerprinted `.npz` caches, enforcing train-only calibration splits (`CAL_SPLITS`), and verifying all outputs through a 34-check audit (`final_validation`), the codebase provides a self-contained implementation of post-hoc selective prediction under distribution shift.

A Function and Class Interface Reference

Table 6: Summary of classes and functions defined in the codebase.

Identifier (Section)	Arguments / Return Type	Responsibility
`set_global_seed` (Sec. 2)	`(seed: int) -> None`	Seeds Python, NumPy, and PyTorch RNGs and sets deterministic flags.
`seed_numpy` (Sec. 2)	`(seed: int) -> Generator`	Returns an isolated NumPy `Generator` seeded with `seed`.
`canonical_json` (Sec. 3)	`(obj) -> str`	Serializes `obj` to key-sorted, compact JSON.
`sha256_text` (Sec. 3)	`(text: str) -> str`	Computes the hex SHA-256 digest of `text`.
`config_fingerprint` (Sec. 3)	`(extra: dict) -> str`	Returns a 16-char SHA-256 fingerprint over core config and `extra`.
`assert_finite` (Sec. 3)	`(name: str, arr: ndarray) -> None`	Raises `ValueError` if `arr` contains non-finite entries.
`validate_probabilities` (Sec. 3)	`(probs: ndarray, atol=1e-5) -> None`	Validates shape $[N, C]$, range $[0, 1]$, finiteness, and row sums $= 1$.
`save_array_cache` (Sec. 3)	`(path, arrays, metadata) -> None`	Atomically writes `.npz` arrays and `.json` metadata.
`load_array_cache` (Sec. 3)	`(path, expected_fp) -> tuple | None`	Loads cached arrays and metadata if present and fingerprint matches.
`save_json` / `load_or_none_json` (Sec. 3)	`(path, obj)` / `(path)`	Writes canonical JSON or safely loads JSON (returning `None` on error).
`_dataset_present` (Sec. 4)	`(root, required_paths) -> bool`	Checks whether all required dataset files exist under `root`.
`download_with_progress` (Sec. 4)	`(url: str, output_path: Path) -> None`	Downloads a URL via a `.part` file with a `tqdm` progress bar.
`validate_tar` (Sec. 4)	`(path: Path) -> bool`	Verifies that `path` is a non-empty, readable tar archive.
`extract_cifar10c_tar` (Sec. 4)	`() -> None`	Ensures all 6 CIFAR-10-C `.npy` files and `labels.npy` are extracted.
`make_calibration_split` (Sec. 5)	`(seed, n_total=50000, n_cal=5000) -> ndarray`	Samples 5,000 unique sorted train indices without replacement.
`build_prompts` / `build_all_prompt_texts` (Sec. 7)	`(class_names, prompt_key)`	Formats CIFAR-10 class names into prompt templates `P0`–`P3`.
`get_text_embeddings` (Sec. 7)	`(prompt_key: str) -> ndarray`	Computes or loads cached $(10, 512)$ L2-normalized CLIP text embeddings.
`ArrayImageDataset` (Sec. 8)	`(images, labels, indices=None)`	Dataset wrapper for memory-mapped CIFAR-10-C numpy arrays.
`TorchvisionImageDataset` (Sec. 8)	`(base_dataset, label_transform=None)`	Dataset wrapper applying CLIP `preprocess` to torchvision datasets.
`cifar10_clean_dataset`, `cifar100_dataset`, `svhn_dataset`, `cifar10c_dataset` (Sec. 8)	Dataset factory functions	Construct preprocessed dataset objects for train/test/OOD/corruption splits.
`encode_image_dataset` (Sec. 9)	`(dataset, cache_name, expected_extra) -> tuple`	Encodes images into cached $(N, 512)$ L2-normalized CLIP embeddings.

Continued from previous page

Identifier (Section)	**Arguments / Return Type**	**Responsibility**
`compute_prompt_logits` (Sec. 9)	`(image_embeddings, prompt_embeddings) -> dict`	Computes scaled cosine similarity logits $(N, 10)$ for each prompt.
`_validate_prompt_logits_bank` (Sec. 9)	`(logits, labels, embeddings, cache_name) -> dict`	Validates prompt keys, array shapes, and finiteness of a logit bank.
`get_condition_logits` (Sec. 9)	`(dataset_key, dataset_obj, cache_name, extra)`	Loads or computes validated `(logits, labels, emb)` for a dataset split.
`softmax_np` (Sec. 11)	`(logits: ndarray) -> ndarray`	Numerically stable row-wise softmax in `float32`.
`entropy_np` (Sec. 11)	`(probs, normalized=False) -> ndarray`	Row-wise Shannon entropy (optionally divided by $\ln C$).
`top12_stats` (Sec. 11)	`(probs: ndarray) -> tuple`	Returns top-1 probability, top-2 probability, and top-1 minus top-2 margin.
`prediction_from_probs` (Sec. 11)	`(probs: ndarray) -> ndarray`	Returns row-wise argmax class indices as `int64`.
`confidence_uncertainties` (Sec. 11)	`(logits: ndarray) -> dict`	Computes probabilities, predictions, MSP, entropy, margin, and uncertainties.
`temperature_scale_logits` (Sec. 11)	`(logits, temperature: float) -> ndarray`	Divides logits by a positive finite scalar temperature.
`fit_temperature` (Sec. 12)	`(logits_cal, labels_cal, bounds) -> float`	Fits scalar temperature T minimizing calibration NLL via bounded search.
`fit_seed_calibration` (Sec. 12)	`(seed: int) -> dict`	Fits $T^{(s)}$ and split-conformal $\hat{q}_{1-\alpha}^{(s)}$ on the seed's calibration split.
`prompt_level_outputs` (Sec. 13)	`(logits_by_prompt, return_probs=False) -> tuple`	Vectorized majority-vote prediction, prompt agreement, and instability.
`conformal_sets` (Sec. 13)	`(probs: ndarray, qhat: float) -> tuple`	Constructs boolean conformal prediction sets and integer set sizes.
`threshold_for_target_coverage` (Sec. 14)	`(confidence_cal, target: float) -> float`	Computes $(1 - \text{target})$ lower quantile threshold on calibration scores.
`selective_metrics` (Sec. 14)	`(confidence, correct, threshold) -> dict`	Computes coverage, selective risk, abstention rate, and answered count.
`max_coverage_under_risk` (Sec. 14)	`(confidence_cal, correct_cal, risk_target) -> float`	Finds lowest tie-consistent threshold with calibration risk $\leq$ risk_target.
`risk_coverage_curve`, `aurc`, `oracle_aurc`, `eaurc` (Sec. 14)	`(confidence, correct)`	Computes prefix risk-coverage curves, trapezoidal AURC, Oracle AURC, and EAURC.
`fpr95`, `ood_metrics` (Sec. 14)	`(id_uncertainty, ood_uncertainty)`	Computes FPR at 95% OOD TPR, AUROC, and AUPR-Out.
`ece_score`, `nll_score`, `brier_score_multiclass` (Sec. 14)	`(probs, labels[, bins])`	Computes Expected Calibration Error, multiclass NLL, and Brier score.

Continued from previous page

Identifier (Section)	Arguments / Return Type	Responsibility
`bootstrap_ci_scalar`, `bootstrap_risk_ci`, `bootstrap_aurc_ci` (Sec. 15)	Bootstrap helper functions	Compute percentile bootstrap CIs for scalar statistics, risk, and AURC.
`evaluate_seed_condition` (Sec. 16)	`(seed, condition_id, logits_by_prompt, ...)`	Evaluates all 6 uncertainty methods for one seed and condition.
`evaluate_condition_experiment` (Sec. 17)	`(exp_row) -> tuple`	Dispatches unexpanded experiment rows to `DATA_BANK`.
`expanded_experiment_matrix` (Sec. 18)	`() -> pd.DataFrame`	Expands `E3`–`E6` across 6 corruptions to form the 30-row matrix.
`get_bank_key` (Sec. 18)	`(row) -> str`	Maps any expanded matrix row to its `DATA_BANK` key.
`metric_at_target` (Sec. 19)	`(summary, target, metric) -> DataFrame`	Aggregates mean, std, and count across seeds at a fixed coverage target.
`bootstrap_risk_ci_progress`, `bootstrap_aurc_ci_progress`, `bootstrap_auroc_ci_progress` (Sec. 20)	Progress-bar bootstrap functions	Compute 2,000-resample 95% CIs with `tqdm` progress bars.
`save_fig`, `plot_risk_coverage`, `reliability_data` (Sec. 21)	Plotting helpers	Render and save publication figures in PNG and PDF formats.
`extract_failure_cases` (Sec. 22)	`(sample_df, n_each=25) -> DataFrame`	Extracts top-25 diagnostic failure records across 6 categories.
`_safe_has_columns`, `_safe_nonempty`, `_safe_finite_columns`, `_safe_unique_rows`, `final_validation` (Sec. 25)	Validation helpers	Execute the 34-check pipeline verification audit.
`copy_tree_flat` (Sec. 26)	`(src_dir: Path, dst_dir: Path) -> int`	Recursively copies generated files into `final_output/`.

B Notebook Cell Inventory and Execution Map

Table 7: Complete cell-by-cell inventory of the 59-cell Jupyter notebook.

Cell	Type	Cell ID	Exec.	Section / Responsibility
00	Markdown	`502d5a7d`	–	Title, project overview, and design invariants.
01	Markdown	`bc336630`	–	Section 1 header: Environment and installation.
02	Code	`f15494b3`	8	`%pip install` commands for dependencies and OpenAI CLIP.
03	Code	`bdd8d85d`	9	Top-level library imports and runtime/GPU environment summary.
04	Markdown	`41f82bd4`	–	Section 2 header: Imports, configuration, and reproducibility.
05	Code	`3d413372`	10	Google Drive mount, `CONFIG`, `PROMPTS`, `EXPERIMENTS`, and `ROOTS`.
06	Code	`0e5cc294`	11	`set_global_seed` and `seed_numpy` initialization.

Continued from previous page

Cell	Type	Cell ID	Exec.	Section / Responsibility
07	Markdown	`b4a0d6cc`	–	Section 3 header: Utility functions.
08	Code	`c48990ba`	12	Fingerprinting, validation, and atomic `.npz`/`.json` caching utilities.
09	Markdown	`43468ca0`	–	Section 4 header: Dataset download, verification, and label definitions.
10	Code	`a8a63e89`	13	CIFAR-10, CIFAR-100, and SVHN dataset presence checks and loading.
11	Code	`58183184`	14	`CIFAR-10-C.tar` download, tar validation, and `.npy` extraction.
12	Code	`3c41d138`	15	CIFAR-10-C shape and test-label alignment verification across severities 1, 3, 5.
13	Markdown	`3012aa9a`	–	Section 5 header: Data splits and leakage guardrails.
14	Code	`c11a5e1e`	16	`make_calibration_split`, `CAL_SPLITS`, `TEST_INDICES` offset check, `split_metadata.json`.
15	Markdown	`5b7e3a96`	–	Section 6 header: Model loading and preprocessing.
16	Code	`d4d2adbe`	17	Loads frozen CLIP `ViT-B/32` and verifies 0 trainable parameters.
17	Markdown	`b150a458`	–	Section 7 header: Prompt construction.
18	Code	`6ccf9063`	18	`build_prompts`, `build_all_prompt_texts`, and `ALL_PROMPT_TEXTS`.
19	Code	`d45d812a`	19	`get_text_embeddings` and cached `TEXT_EMBEDDINGS` dictionary.
20	Markdown	`c35c53a0`	–	Section 8 header: Dataset adapters.
21	Code	`f38c63d6`	20	`ArrayImageDataset`, `TorchvisionImageDataset`, and dataset factory functions.
22	Markdown	`0ce4b653`	–	Section 9 header: Efficient image inference and compatible cache.
23	Code	`b709b243`	21	`encode_image_dataset`, `compute_prompt_logits`, `_validate_prompt_logits_bank`, and `get_condition_logits`.
24	Markdown	`1d1f0783`	–	Section 10 header: Load/cache the reusable inference bank.
25	Code	`3fa21cc8`	22	Constructs and validates `CALIBRATION_BANK` and all 21 `DATA_BANK` entries.
26	Markdown	`fb12ac61`	–	Section 11 header: Stable probability, confidence, and uncertainty functions.
27	Code	`985fdf57`	23	`softmax_np`, `entropy_np`, `top12_stats`, `confidence_uncertainties`, and `temperature_scale_logits`.
28	Markdown	`41e613b6`	–	Section 12 header: Temperature calibration on development data only.
29	Code	`5793b62c`	26	`fit_temperature`, `fit_seed_calibration`, `SEED_CALIBRATIONS`, and `seed_calibration_parameters.json`.
30	Markdown	`2bfcf765`	–	Section 13 header: Prompt agreement, instability, and conformal prediction sets.
31	Code	`9675b85b`	27	Vectorized `prompt_level_outputs` and `conformal_sets`.
32	Markdown	`e4bfd929`	–	Section 14 header: Selective prediction, risk-coverage, AURC/EAURC, and OOD metrics.

Continued from previous page

Cell	Type	Cell ID	Exec.	Section / Responsibility
33	Code	32ba482b	28	Selective, risk-coverage, AURC/EAURC, OOD, and calibration metric functions.
34	Markdown	80c6e447	–	Section 15 header: Bootstrap confidence intervals.
35	Code	41914806	29	`bootstrap_ci_scalar`, `bootstrap_risk_ci`, and `bootstrap_aurc_ci`.
36	Markdown	52e3616c	–	Section 16 header: Per-condition computation.
37	Code	9b7f001d	30	`METHODS` list and `evaluate_seed_condition`.
38	Markdown	51146088	–	Section 17 header: Correct calibration reuse and per-sample record generation.
39	Code	cd58f04e	35	Precomputes `CAL_FEATURES`, defines `RESULT_COLUMNS` and `evaluate_condition_experiment`.
40	Markdown	bd97936a	–	Section 18 header: Experiment matrix expansion and full execution.
41	Code	4c314772	36	`expanded_experiment_matrix` and 30-row `FULL_MATRIX`.
42	Code	7e7211e4	38	Full experiment loop writing `all_results.csv` and `per_sample_records.csv`.
43	Markdown	128c9e92	–	Section 19 header: Derived tables and primary statistical summaries.
44	Code	cd33f404	39	`metric_at_target` and export of 8 CSV summary tables.
45	Markdown	b6f553f6	–	Section 20 header: Bootstrap statistical analysis.
46	Code	1f6952f7	41	Progress-aware 2,000-resample bootstrap CIs writing `bootstrap_95ci.csv`.
47	Markdown	0e2ff856	–	Section 21 header: Publication-quality visualizations.
48	Code	351e587c	42	Generates 13 PNG and 13 PDF figures under `figures/`.
49	Markdown	fbb67bb0	–	Section 22 header: Failure-case extraction.
50	Code	c3b3ddaf	43	`extract_failure_cases` exporting 6,150 rows to `failure_cases.csv`.
51	Markdown	f9378916	–	Section 23 header: Main, robustness, risk, calibration, OOD, and statistical tables.
52	Code	274ea952	44	Exports `main_method_comparison.csv`, `robustness_table.csv`, and 8 Markdown tables.
53	Markdown	ba274901	–	Section 24 header: Metadata, configuration, and result provenance.
54	Code	4830c946	45	Writes `experiment_metadata.json`.
55	Markdown	378d609b	–	Section 25 header: Integrated final validation and completeness audit.
56	Code	2a7ac5bd	48	`final_validation()` 34-check audit and `final_validation_checklist.csv`.
57	Markdown	1b6659da	–	Section 26 header: Final export and automatic downloads.
58	Code	820406c0	49	Copies deliverables to `final_output/`, writes `completion_summary.json`, and builds ZIPs.